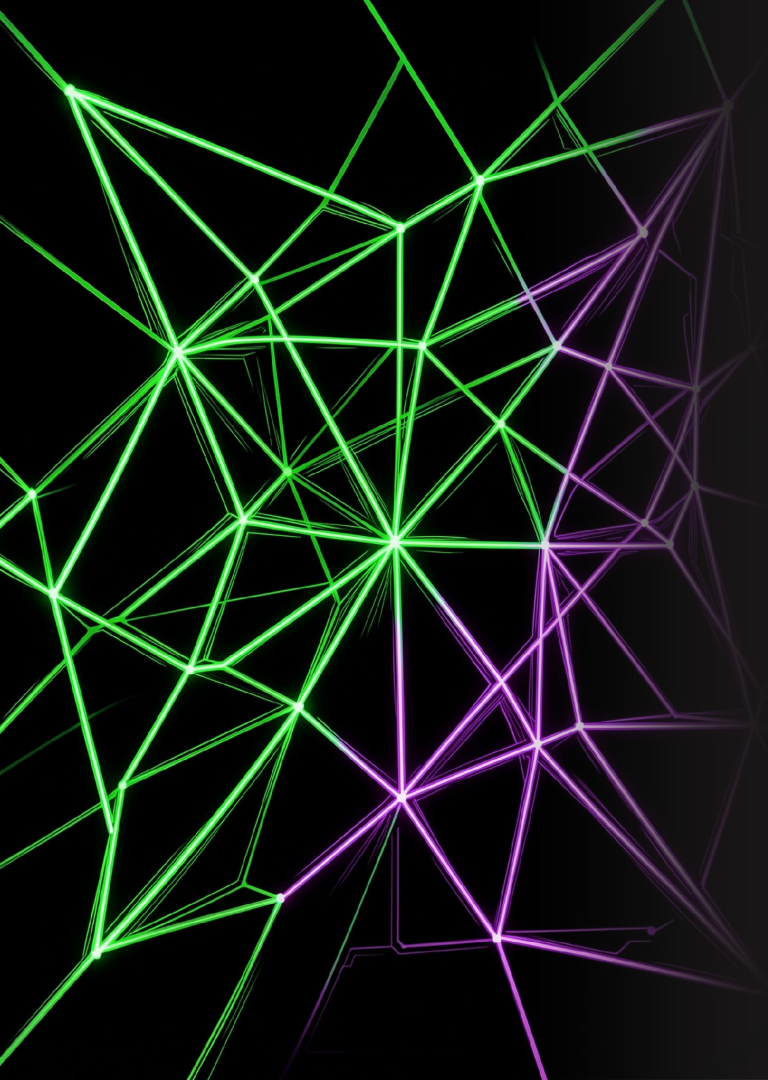




Spring AI: Construindo Agentes Cognitivos Multimodais

Detecção de Fraude em Tempo Real com Java e IA

Stack

Java 21, Spring Boot, Spring AI, pgvector, Gemini, Langfuse

Repositório: [workshop-spring-ai-wtec-2026-1](https://github.com/workshop-spring-ai-wtec-2026-1)

Objetivo

Ir além do CRUD tradicional e implementar sistemas autônomos.

Instrutor: Joel Maykon

Função

Analista Plataforma Baixa Sênior na Caixa Econômica Federal.

Comunidade

Contribuidor do Java Day e divulgador de IA usando Java.

Objetivo

Integrar IA em aplicações Java corporativas.



Segurança Avançada: Como Identificar e Mitigar Fraudes Além do CRUD

Limite das Regras

Usar apenas condições (if/else) pode facilitar para os fraudadores.

Paradigma de Agentes

O foco é ir além do CRUD passivo para sistemas mais autônomos.

A Mudança: Ir além do CRUD passivo. É preciso migrar para sistemas autônomos e proativos.

Cognição (LLM)

O modelo que **analisa** o contexto e **decide** o nível de risco.

Memória (Vector)

Armazena dados do usuário e **identifica** anomalias.

Ação (Tools)

Permite ações **imediatas** em aplicações **Java** para bloquear contas ou disparar alertas.

Como o Antifraude Julga:

Dados, Imagem, Voz



Metadados (JSON)

Valores, contas envolvidas e localizações da transação.



Visão Computacional

Deteção de alterações gráficas em fotos de comprovantes.



Análise de voz

Transcrição e análise de coação ou voz sintética (deepfake).



Consolidação

O agente cruza as informações e emite uma decisão única.



Stack Tecnológica e Infraestrutura

A arquitetura utiliza tecnologias modernas do ecossistema Java e ferramentas de IA

Stack

Java 21 (Virtual Threads), Spring AI (Orquestração), pgvector (busca vetorial), Gemini (LLM).

Infraestrutura

Docker Compose (Postgres, MinIO S3 e Kafka).

● PAUSA PARA PRÁTICA (Etapa 1 Hands-on):

Atividade: Faça o fork e clone do repositório: github.com/joelmaykon94/workshop-spring-ai-wtec-2026-1, entre no diretório do projeto, rode docker compose up -d localmente e certifique-se de que os containers estão ativos antes de iniciarmos a mão na massa.

Implementando Busca Vetorial (RAG) com pgvector

01

Conceito do RAG

Fornecer dados para o LLM como contexto relevante no prompt.

02

pgvector

Os vetores das tentativas históricas de fraude.

03

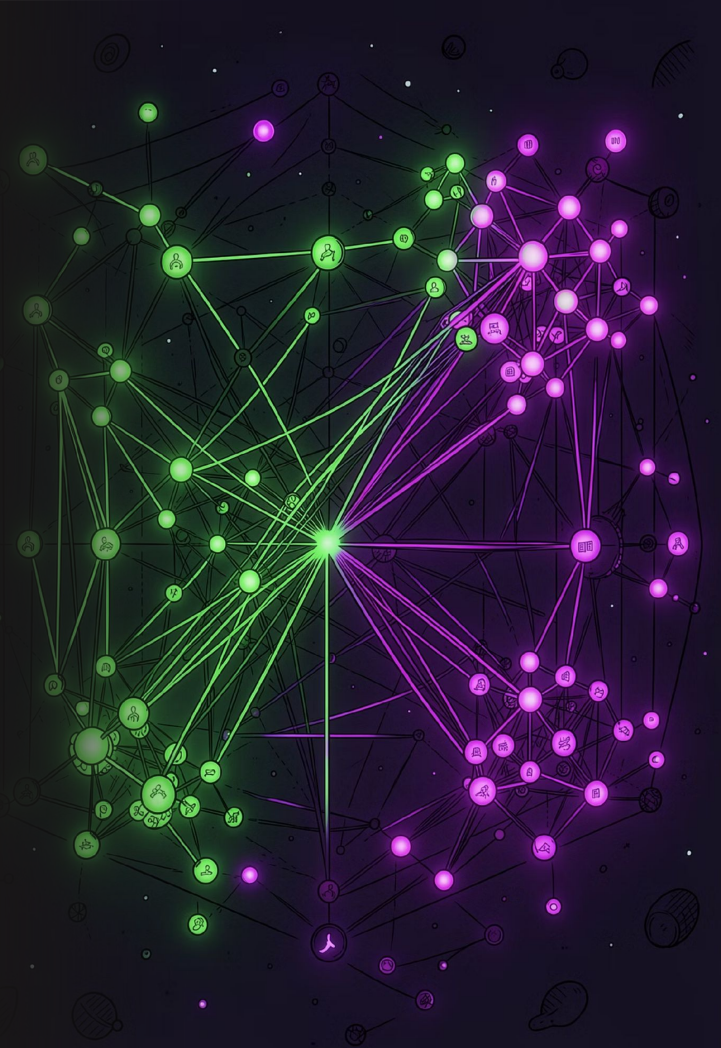
Verificação

Comparação semântica instantânea da transação com a base histórica.



PAUSA PARA PRÁTICA (Etapa 2 Hands-on):

Atividade: Injetar e configurar o PgVectorVectorStore e escrever o método de busca por similaridade.



Programando o ChatClient, Tools e Outputs Estruturados

Configuração do ChatClient

Adicionando advisors de histórico e RAG ao cliente.

Function Calling (Tools)

Métodos expostos para consultar saldos ou bloquear contas.

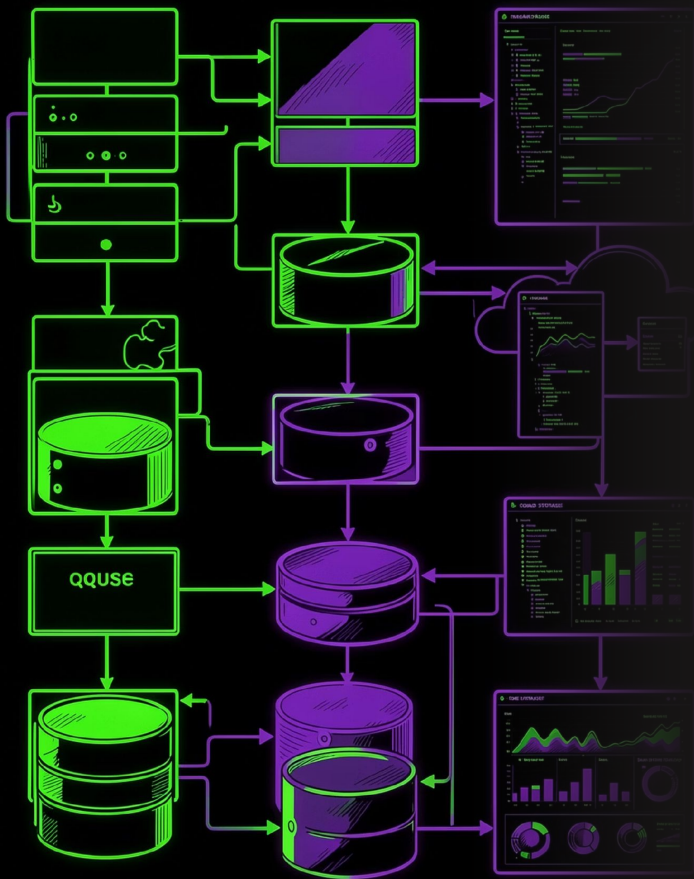
Saída Estruturada

Mapeamento para o Record Java (FraudAssessment).



PAUSA PARA PRÁTICA (Etapa 3 Hands-on):

Atividade: Programar o Agente para integrar os advisors, as tools e retornar a análise mapeada para o Record.



Arquitetura para Produção & Observabilidade com Langfuse

Processamento Kafka

Risco processado de forma reativa e distribuída.

Storage (MinIO)

Upload assíncrono de cupons e áudio para buckets S3.

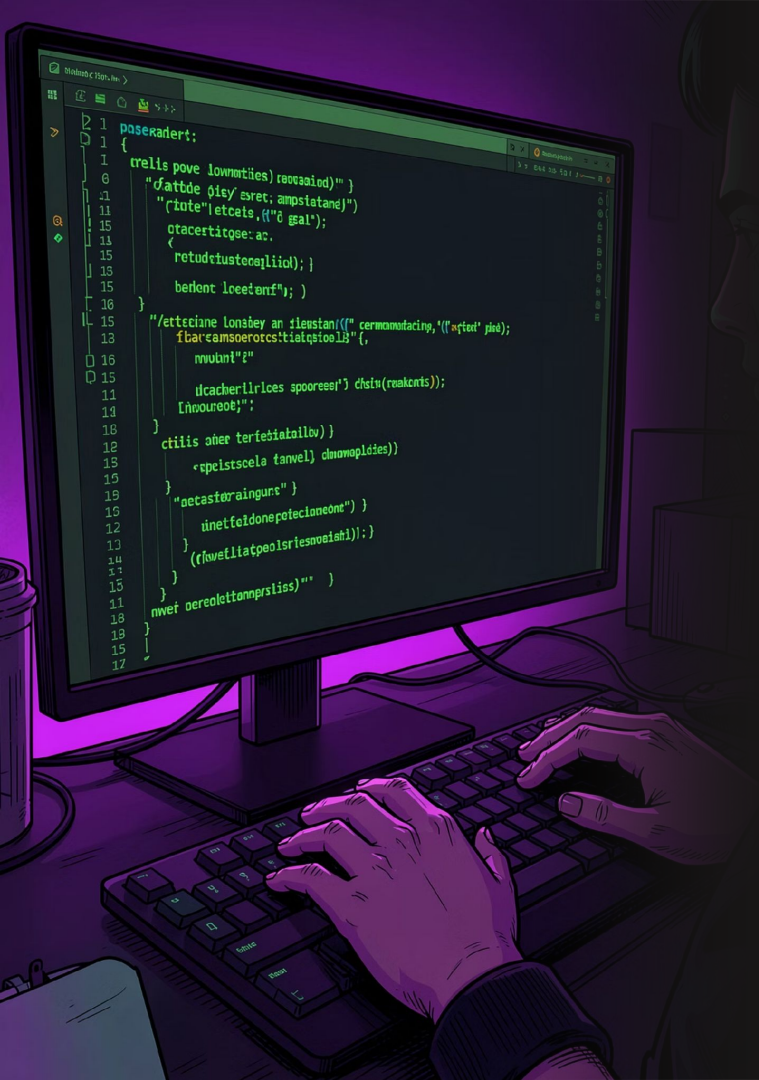
Observabilidade (Langfuse)

Tracing de prompts, custo de tokens e latências de execução.



PAUSA PARA PRÁTICA (Etapa 4 Hands-on):

Atividade: Configurar as chaves de acesso na aplicação Spring e depurar chamadas de IA.



Roteiro Prático de Codificação

01

Comandos do Terminal

`docker compose logs -f`
`mvn spring-boot:run`

03

Branch de Suporte

A branch `solucao` contém o código 100% pronto para consulta.

02

Como Codificar

Procurar por comentários `TODO` no projeto.

Dúvidas e Conexão Profissional

Resumo

Construímos um agente multimodal, seguro, de baixa latência e observável usando ecossistema Spring Boot.

Discursão sobre os trade-offs

Latência, custo financeiro, alucinações, LGPD e experiência.

Networking

LinkedIn: joelmaykon / GitHub: joelmaykon94

